

ANÁLISE

TÉCNICA

**PORTABILIDADE MOBILE
COM SUPORTE OFFLINE**

1. CONTEXTO E OBJETIVO

Este documento compara as possíveis formas de levar o sistema de gestão (previsto para a versão Site) para uso em dispositivos móveis com suporte a operação offline, avaliando alternativas ao desenvolvimento de um aplicativo nativo tradicional. O objetivo é apontar a solução mais adequada considerando que o sistema é de uso interno da empresa, com foco em reaproveitar ao máximo o código e o esforço já investidos na versão Site.

2. SOLUÇÕES POSSÍVEIS		
ITEM	SOLUÇÃO	DESCRIÇÃO
1	App Nativo (React Native)	Aplicativo mobile desenvolvido em React Native, publicado como um app instalável, com banco de dados local (SQLite/WatermelonDB) para uso offline.
2	PWA (Progressive Web App)	O próprio site é transformado em um app instalável direto do navegador, usando Service Worker (cache) e IndexedDB (dados locais) para funcionar offline — sem projeto separado.
3	Capacitor / TWA (híbrido)	O PWA é “embrulhado” em uma casca nativa para gerar um instalável (.apk), podendo ser publicado em loja, mantendo o mesmo código do Site.

3. VANTAGENS E DESVANTAGENS		
SOLUÇÃO	VANTAGENS	DESVANTAGENS
App Nativo (React Native)	• Melhor desempenho e acesso mais completo a recursos do dispositivo (câmera, notificações push nativas). • Experiência de uso mais “polida”. • Pode ser publicado na Play Store.	• Projeto de desenvolvimento separado do Site (mesmo reaproveitando lógica/tipos). • Atualizações dependem da instalação de nova versão pelo usuário. • Maior esforço e custo de manutenção contínuo.
PWA	• Reaproveita 100% do código React já usado no Site — não é um projeto separado. • Atualização instantânea (basta publicar no servidor). • Instalação simples direto do navegador, sem loja. • Menor custo e prazo de desenvolvimento.	• Acesso mais limitado a recursos nativos (ex.: push tem suporte variável entre navegadores). • Em casos raros, dados locais podem ser limpos pelo navegador por falta de uso/espço. • Experiência um pouco menos “nativa”.
Capacitor / TWA (híbrido)	• Mantém o mesmo código do PWA. • Permite gerar um instalável (.apk) e publicar em loja, se necessário no futuro. • Meio-termo entre reaproveitamento e distribuição formal.	• Adiciona uma camada extra de configuração e build. • Não traz ganho real de desempenho sobre o PWA puro. • Só compensa se realmente for necessário publicar em loja.

4. Melhor opção recomendada: PWA (Progressive Web App)

Para este projeto — sistema de uso interno, sem necessidade de publicação em loja — o PWA é a opção recomendada. Ele reaproveita integralmente o código já desenvolvido para a versão Site, elimina a manutenção de um projeto mobile separado, permite atualizações instantâneas e atende ao requisito de uso offline por meio de Service Worker (cache de arquivos) e IndexedDB (dados locais). Caso no futuro surja a necessidade de publicar o aplicativo em loja ou acessar recursos nativos mais avançados, o mesmo PWA pode ser embrulhado com Capacitor/TWA, sem necessidade de reescrever o sistema.

5. Cronograma da melhor opção (PWA)

Este cronograma considera que a versão Site já está finalizada e estável antes do início desta etapa. Teremos checkPoints semanais no qual será entregue:

1. Semana 1:

- a. Configuração do Service Worker e do manifest.json (ícones, nome, tela cheia) e definição da estratégia de cache dos arquivos estáticos do Site.

2. Semana 2:

- a. Modelagem do IndexedDB para as entidades espelhadas (clientes, catálogo de serviços e peças, pedidos) e camada de acesso aos dados locais.

3. Semana 3:

- a. Telas de consulta funcionando via cache local — clientes, catálogo e agenda do dia.

4. Semana 4:

- a. Criação/edição de pedidos offline e fila de sincronização das alterações locais.

5. Semana 5:

- a. Envio da fila de sincronização quando a conexão retorna, resolução de conflitos (last write wins) e alteração de status do pedido.

6. Semana 6:

- a. Testes em diferentes navegadores/dispositivos, ajuste do fluxo de instalação (“adicionar à tela inicial”) e publicação da versão PWA.